PROJECT REPORT: TrafficGuard Ultra

AI-Powered Adaptive Traffic Management System

Submitted by: [Your Name]

Department: Computer Science & Engineering

Subject: Major Project / Minor Project

📑 TABLE OF CONTENTS

1. **Chapter 1: Introduction**
    - 1.1 Abstract
    - 1.2 Problem Statement
    - 1.3 Objectives
2. **Chapter 2: System Analysis & Design**
    - 2.1 Proposed Architecture
    - 2.2 Data Flow Diagram (DFD)
    - 2.3 Technology Stack
3. **Chapter 3: Implementation Details**
    - 3.1 Vision Module (YOLOv8 & OpenCV)
    - 3.2 Logic Controller (Adaptive Algorithm)
    - 3.3 Frontend Dashboard (React.js)
4. **Chapter 4: Key Features & Unique Selling Points**
    - 4.1 Emergency Vehicle Priority (EVP)
    - 4.2 Stalled Vehicle Detection
    - 4.3 Night Vision Mode
5. **Chapter 5: User Manual & Testing**
    - 5.1 Installation Guide
    - 5.2 Demonstration Triggers
6. **Chapter 6: Conclusion & Future Scope**

CHAPTER 1: INTRODUCTION

1.1 Abstract

Traffic congestion is one of the most critical issues in modern urban infrastructure. Traditional traffic control systems operate on **Fixed-Cycle Timers** (e.g., 60 seconds for Green, regardless of traffic volume). This approach is inefficient, leading to wasted fuel, increased pollution, and valuable time loss.

TrafficGuard Ultra is a smart, automated traffic management system that uses **Computer Vision** and **Artificial Intelligence** to regulate traffic signals in real-time. By utilizing the **YOLOv8** object detection model, the system analyzes live camera feeds to count vehicles and allocates green signal duration dynamically. Furthermore, the system incorporates advanced safety features such as Ambulance Priority, Obstacle Detection, and Night Mode, making it a complete Smart City solution.

1.2 Problem Statement

- **Static Timers:** A lane with 0 cars waits for 60 seconds, while a lane with 50 cars gets the same time.
- **Emergency Delays:** Ambulances often get stuck at red lights, leading to critical delays in medical response.
- **Lack of Monitoring:** Authorities cannot detect stalled vehicles or accidents remotely in real-time.
- **Night Inefficiency:** Bright interface glares and lack of adaptability during low-light conditions.

1.3 Objectives

1. To develop a system that detects vehicle density using a camera.
2. To implement an algorithm that adjusts Green Light timing based on the number of vehicles (Adaptive Timing).
3. To provide a "Green Corridor" automatically for Emergency Vehicles (Ambulances).
4. To create a centralized Dashboard for traffic police to monitor and control signals manually.

CHAPTER 2: SYSTEM ANALYSIS & DESIGN

2.1 Proposed Architecture

The system is divided into three functional layers: **Sensing (Input)**, **Processing (Logic)**, and **Visualization (Output)**.

```
graph TD
    subgraph "Input Layer (Vision)"
    A[Webcam / CCTV Feed] -->|Raw Frames| B[OpenCV Processing]
    end

    subgraph "Processing Layer (Backend)"
    B -->|Pre-processed Image| C[YOLOv8 AI Model]
    C -->|Object Detection| D{Decision Engine}
    D -->|Vehicle Count| E[Adaptive Timer Logic]
    D -->|Class: Cell Phone| F[Emergency Trigger]
    D -->|Class: Bottle| G[Obstacle Trigger]
    D -->|Brightness < 60| H[Night Mode Trigger]
    end

    subgraph "Output Layer (Frontend)"
    E & F & G & H -->|WebSocket JSON Payload| I[React.js Dashboard]
    I -->|Admin Commands| D
    end
```

2.2 Data Flow Explanation

1. **Image Capture:** The system captures video frames from a live camera source.
2. **Segmentation:** The screen is virtually divided into 4 Quadrants (North, South, East, West) to simulate a 4-way junction.
3. **Detection:** YOLOv8 scans the frame to identify objects (Cars, Bikes, Trucks, Ambulances).
4. **Logic Execution:**
   - If an **Ambulance** is found → Override Timer → Set Green immediately.
   - Else → Count vehicles → Assign time slot (Low/Med/High).
5. **Transmission:** The computed data (Counts, Images, Timer status) is sent to the Frontend via WebSockets.

2.3 Technology Stack

- **Programming Language:** Python 3.9+ (Backend), JavaScript (Frontend).
- **AI Model:** Ultralytics **YOLOv8n** (Nano version for high-speed CPU inference).
- **Image Processing: OpenCV** (cv2) for frame manipulation and brightness calculation.
- **Communication Protocol: WebSockets** (ws://) for low-latency real-time updates.
- **Frontend Framework: React.js** (Vite) for a component-based, responsive UI.
- **Styling: Tailwind CSS** for modern, dark-themed aesthetics.

CHAPTER 3: IMPLEMENTATION DETAILS

3.1 Vision Module (YOLOv8 & OpenCV)

We use the **COCO Dataset** pre-trained model. To make the project feasible for a prototype, we map specific objects to real-world traffic scenarios:

- **Real World:** Ambulance ➔ **Prototype:** Mobile Phone (Class ID 67).
- **Real World:** Stalled Vehicle/Rock ➔ **Prototype:** Water Bottle (Class ID 39).

The video frame is resized to `640x480` for optimization. OpenCV calculates the **HSV Brightness Value**; if the mean V-value drops below 60, the system flags `is_night = True`.

3.2 Logic Controller (The "Brain")

The core logic resides in a State Machine inside `traffic_backend.py`.

Algorithm for Adaptive Timing:

```
IF Emergency_Detected:
    Current_State = EMERGENCY
    Timer = 15s (Priority)
    Active_Light = Green (For Emergency Lane)

ELSE IF Manual_Override:
    Control passes to Admin (React Button)

ELSE (Auto Mode):
    Check Vehicle Count (N) in next lane:
    - IF N < 5  : Timer = 10s (Low Traffic)
    - IF N < 10 : Timer = 20s (Medium Traffic)
    - IF N < 15 : Timer = 30s (High Traffic)
    - IF N > 15 : Timer = 45s (Traffic Jam)
```

3.3 Frontend Dashboard (React.js)

The dashboard is designed as a **Control Room Interface**. It features:

- **Live Grid:** 4 split-screen camera feeds showing traffic in real-time.
- **Central HUD:** A heads-up display showing the active timer and signal state.
- **Sidebar:** Contains Analytics (Pie charts for vehicle types) and Admin Control buttons.
- **Alert System:** Visual pop-ups (Red/Blue borders) for Obstacles and Ambulances.

CHAPTER 4: KEY FEATURES & USP

4.1 🚑 Emergency Vehicle Priority (EVP)

This is the most critical safety feature.

- **Function:** Uses AI to identify emergency vehicles visually.
- **Process:** Upon detection, the current cycle is interrupted. The emergency lane gets a Green signal, and all others turn Red.
- **Visual Indicator:** The dashboard pulses **BLUE**, and an "EMERGENCY OVERRIDE" banner appears.

4.2 🛑 Stalled Vehicle Detection (Obstacle Alert)

Detects stationary objects that shouldn't be on the road.

- **Scenario:** A car breaks down or a rock falls on the road.
- **Function:** If the object (simulated by a Bottle) is detected in a green zone, the system alerts the admin.
- **Visual Indicator:** The camera feed border turns **ORANGE** with a "WARNING" tag.

4.3 🌙 Intelligent Night Mode

Traffic lights operate differently at night (often blinking yellow or sensor-based).

- **Function:** Automatically detects sunset/low light via camera sensor.
- **UI Change:** The Dashboard switches to a high-contrast dark mode (OLED Black) to reduce eye strain for operators.
- **System Change:** Vision sensitivity is adjusted to handle low light better.

CHAPTER 5: USER MANUAL & TESTING

5.1 Installation & Setup

Prerequisites: Python 3.10+, Node.js 18+.

1. **Backend Setup:**

   ```
   pip install opencv-python ultralytics websockets numpy
   python traffic_backend.py
   ```

2. **Frontend Setup:**

```
cd frontend
npm install
npm run dev
```

3. **Access:** Open `http://localhost:5173` in your browser.

5.2 Demonstration Guide (How to present)

When presenting to the examiner, use the following "Triggers" to demonstrate features since we cannot bring real cars and ambulances to the lab.

Feature To Show	Action (Trigger)	What Happens on Screen?
1. Adaptive Timer	Stand in front of the camera or move hands.	The timer will set to **20s** or **30s** depending on how many "objects" (people) are detected.
2. Ambulance Logic	Show a **Cell Phone** clearly to the camera.	The system shouts **"PRIORITY"**, timer resets, and the dashboard turns **Blue**.
3. Stalled Car	Place a **Water Bottle** on the table/frame.	An **Orange Alert** ("STALLED VEHICLE") appears on that specific camera feed.
4. Night Mode	Cover the camera lens with your palm.	The UI turns Pitch Black, and a **Moon Icon** appears.
5. VIP Movement	Click **"FORCE NORTH"** on the website.	The North signal locks to **GREEN** indefinitely until you click "AUTO".

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Conclusion

TrafficGuard Ultra successfully demonstrates how AI can replace outdated timer-based systems. By processing visual data in real-time, we achieved optimized traffic flow, reduced waiting times for empty lanes, and most importantly, provided a lifesaving corridor for ambulances. The integration of React and Python creates a seamless, user-friendly experience for traffic controllers.

6.2 Future Scope

1. **Multi-Junction Synchronization:** Using Cloud (AWS/Firebase) to sync multiple traffic lights so vehicles get a "Green Wave."
2. **E-Challan System:** Integrating Automatic Number Plate Recognition (ANPR) to fine drivers who jump signals.
3. **Pedestrian Safety:** Using the same camera to detect jaywalkers and warn drivers.
4. **Hardware Implementation:** Deploying this logic on edge devices like **Raspberry Pi** or **NVIDIA Jetson Nano** for a standalone roadside unit.

End of Report